Hexagonal Architecture – Ports & Adapters

Clean Architecture Applied to Neurodialogue

Table des matières

1 What is Hexagonal Architecture ? 2
- 1.1 Definition . . . 2
- 1.2 The Core Principle : The Dependency Rule . . . 2
- 1.3 The Two Fundamental Concepts . . . 2
  - 1.3.1 Ports . . . 2
  - 1.3.2 Adapters . . . 2
- 1.4 The 4 Layers of Neurodialogue . . . 3

2 The 6 Ports of Neurodialogue 3

3 Advantages of Hexagonal Architecture for Neurodialogue 3
- 3.1 Advantage 1 – Independence from AI Models . . . 3
- 3.2 Advantage 2 – Unit Tests Without Loading AI Models . . . 4
- 3.3 Advantage 3 – Separation of Django / FastAPI . . . 4
- 3.4 Advantage 4 – Replacement of ChromaDB . . . 4
- 3.5 Advantage 5 – Ease of Debugging . . . 4
- 3.6 Advantage 6 – Team Evolution . . . 5
- 3.7 Advantage 7 – SOLID Principles Compliance . . . 5

4 Comparison : With and Without Hexagonal Architecture 5

5 Actual Code Structure 5

6 Neurodialogue as a Microservice Architecture 6
- 6.1 Microservice Architecture Overview . . . 6
- 6.2 Communication with External Frameworks . . . 6
- 6.3 Internal Architecture (Within the Microservice) . . . 6
- 6.4 Benefits of This Architecture . . . 6
- 6.5 Example Integration with Different Frameworks . . . 8
- 6.6 API Contract . . . 8

7 Verification Rule 8

8 Conclusion 9

1 What is Hexagonal Architecture ?

1.1 Definition

Hexagonal Architecture (also called Ports & Adapters or Clean Architecture) is an architectural style introduced by Alistair Cockburn in 2005. Its fundamental principle is to place the business domain at the center and completely isolate the application code from any external technology (databases, frameworks, APIs, user interfaces).

1.2 The Core Principle : The Dependency Rule

The golden rule is : dependencies can only point inward. The domain knows nothing about the adapters. The adapters know the domain, but the domain does not know them.

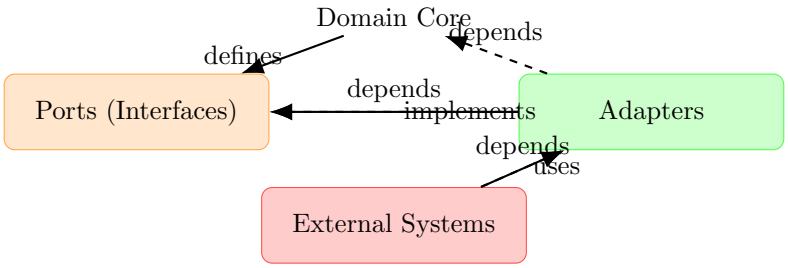


FIGURE 1 – Hexagonal Architecture Dependency Flow

1.3 The Two Fundamental Concepts

1.3.1 Ports

A port is an abstract interface (Python abstract class) that the domain defines to express what it needs. The domain does not know how this need will be satisfied – only what it expects.

Listing 1 – domain/ports/emotionPort.py

```python
from abc import ABC, abstractmethod
from domain.valueObjects.emotionResult import EmotionResult

class EmotionPort(ABC):
    """
␣␣␣␣Port␣defined␣by␣the␣domain.
␣␣␣␣The␣domain␣requires␣that␣someone␣can␣detect␣emotions.
␣␣␣␣It␣does␣not␣know␣if␣it's␣BERT,␣GPT␣or␣a␣manual␣rule.
␣␣␣␣"""

    @abstractmethod
    def detectEmotions(self, text: str, top_k: int = 3) -> EmotionResult:
        pass
```

1.3.2 Adapters

An adapter is a concrete class that implements a port. It connects the domain to a specific technology. It can be replaced without touching the domain.

Listing 2 – adapters/analysis/emotionDetectionAgent.py

```python
from domain.ports.emotionPort import EmotionPort
from domain.valueObjects.emotionResult import EmotionResult
from transformers import AutoTokenizer, AutoModelForSequenceClassification

class EmotionDetectionAgent(EmotionPort):  # implements the port
    """
```

```
    Concrete adapter: uses BERT to detect emotions.
    If we want to change the model, we create a new adapter.
    The domain and the orchestrator remain unchanged.
    """

    def detectEmotions(self, text: str, top_k: int = 3) -> EmotionResult:
        # BERT-specific implementation
        tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
        model = AutoModelForSequenceClassification.from_pretrained("bert-base-
            ↪ uncased")
        # ... emotion detection logic ...
        return EmotionResult(emotions=["joy"], emotionsWithScore={"joy": 0.9})
```

1.4 The 4 Layers of Neurodialogue

Layer	Package	Responsibility and Rule
DOMAIN	domain/	Zero external dependencies. Contains ports, entities, value objects, enums, services.
ADAPTERS	adapters/	Implements ports. Knows ML libraries but not Django/FastAPI.
APPLICATION	application/	Orchestrates domain + adapters through ports. Never touches adapters directly.
PRESENTATION	presentation/	Exposes the application to the outside world (HTTP, FastAPI).

2 The 6 Ports of Neurodialogue

Neurodialogue defines 6 ports in `domain/ports/`. Each port represents a capability that the domain needs.

Port	Contract	Adapter (Implementation)
EmotionPort	Detect emotions in text	EmotionDetectionAgent (BERT) – exposed as FastAPI microservice
IntentAndNeedsPort	Detect intentions and needs	IntentNeedsAgent (BART) – exposed as FastAPI microservice
GenerationPort	Generate an empathic response	CloudBasedGenerator (OpenAI) – external API
RetrievalPort	Retrieve relevant chunks	QueryRetriever (MiniLM) – exposed as FastAPI microservice
VectorStorePort	Store and search vectors	ChromaDBVectorStore – database
VerificationPort	Verify response quality	Empathy/Appropriateness/SafetyVerificator – composite microservice

The `application/chatOrchestrator.py` only calls these 6 ports. It does not know BERT, BART, ChromaDB, or OpenAI. Each machine learning model is encapsulated in its own FastAPI microservice, allowing independent deployment and scaling.

If tomorrow we replace BERT with another emotion model, only the adapter changes – the orchestrator remains untouched.

3 Advantages of Hexagonal Architecture for Neurodialogue

3.1 Advantage 1 – Independence from AI Models

Machine learning models are volatile implementation details. BERT can be replaced by a more efficient model, or BART by a multilingual model better suited to the Tunisian dialect.

Monolithic Architecture : The orchestrator code directly calls `BertTokenizer.from_pretrained()` and `BartForSequenceClassification`. Changing the model requires modifying the orchestrator, tests, and all places where the model is used.

Hexagonal Architecture : The orchestrator only calls `emotion_port.detectEmotions(text)`. To change the model, we create a new adapter that implements `EmotionPort`. The `ChatOrchestrator` doesn't change a single line. Each model is deployed as a separate FastAPI microservice.

3.2 Advantage 2 – Unit Tests Without Loading AI Models

Loading BERT takes 3–5 seconds. BART takes 5–8 seconds. Without hexagonal architecture, every unit test would have to wait for this loading.

Listing 3 – Testing the orchestrator with a mock

```python
from unittest.mock import Mock
from domain.valueObjects.emotionResult import EmotionResult
from application.chatOrchestrator import ChatOrchestrator

def test_orchestrator_without_loading_bert():
    # Create a mock that simulates EmotionPort
    mock_emotion_port = Mock()
    mock_emotion_port.detectEmotions.return_value = EmotionResult(
        emotions=["joy"], emotionsWithScore={"joy": 0.9}
    )

    # The orchestrator receives the mock via dependency injection
    # BERT is never loaded     the test runs in milliseconds
    orchestrator = ChatOrchestrator(emotion_port=mock_emotion_port, ...)
    result = orchestrator.process("I'm␣happy␣today")
    assert result is not None
```

Thanks to ports, Neurodialogue's 30 unit tests run without loading any ML models, making the test suite fast and executable in CI/CD environments. Each AI model is mocked at the port boundary.

3.3 Advantage 3 – Separation of Django / FastAPI

Neurodialogue uses two frameworks : Django for the web and FastAPI for AI. Hexagonal architecture guarantees that the domain depends on neither.

- If we migrate Django to another technology (node js, spring boot), the domain and adapters remain unchanged
- If we migrate FastAPI to a gRPC service, only the `presentation/` layer changes
- The `ChatOrchestrator` works identically whether the call comes from an HTTP router, a test, or a command line
- Each AI model (BERT, BART, MiniLM) runs as an independent FastAPI microservice with its own API contract

3.4 Advantage 4 – Replacement of ChromaDB

ChromaDB is a vector database under active development. If a better alternative appears (Pinecone, Weaviate, pgvector), the replacement is trivial thanks to `VectorStorePort`.

Action	**Files Modified**
Replace ChromaDB with Pinecone	Create `PineconeVectorStore` implementing `VectorStorePort` Modify `systemComponents.py` to inject the new adapter The `ChatOrchestrator`, domain, tests : zero modifications

3.5 Advantage 5 – Ease of Debugging

When a bug appears in the pipeline, hexagonal architecture immediately locates the problem. Each layer has a single responsibility.

Symptom	**Suspected Layer**	**Package to Examine**
Wrong emotion detected	Adapter	`adapters/analysis/`
Blocking pipeline	Application	`application/chatOrchestrator.py`
Incorrect HTTP response	Presentation	`presentation/routers/`
Wrong verification score	Adapter	`adapters/verification/`
Wrong value type	Domain	`domain/valueObjects/`

3.6 Advantage 6 – Team Evolution

In an academic or professional context, multiple developers can work in parallel without conflicts :
— Developer A : improves the `SafetyVerificator` without touching `ChatOrchestrator`
— Developer B : optimizes the `QueryRetriever` independently
— Developer C : adds a new FastAPI router without modifying the agents
— Developer D : deploys a new version of the BERT microservice independently
— With fixed interfaces (ports), each developer works in their own layer

3.7 Advantage 7 – SOLID Principles Compliance

Hexagonal architecture naturally applies SOLID principles to the project :

Principle	**Letter**	**Application in Neurodialogue**
Single Responsibility	S	Each adapter does one thing (BERT = emotions)
Open/Closed	O	Add an adapter without modifying existing code
Liskov Substitution	L	Any adapter can replace its port without breaking the pipeline
Interface Segregation	I	6 specialized ports rather than one generic interface
Dependency Inversion	D	Orchestrator depends on ports (abstractions), not adapters

4 Comparison : With and Without Hexagonal Architecture

Scenario	**Without Hexagonal**	**With Hexagonal**
Change emotion model	Modify orchestrator, tests, imports everywhere	Create a new adapter (1 file)
Test without GPU	Impossible (BERT loads)	Replace with mock (a few lines)
Migrate from ChromaDB	Massive refactoring	New VectorStorePort adapter
Add a 4th verifier	Modify the pipeline	Implement VerificationPort
Debug a bug	Search through all the code	Locate the affected layer
Unit tests	Slow (model loading)	Fast (mocks via ports)
Model deployment	Monolithic, coupled	Independent FastAPI microservices

5 Actual Code Structure

The current project structure perfectly respects the hexagonal architecture after moving `ports/` into `domain/` :

```
neurodialogue/
 domain/                  # Pure business objects
    ports/               # 6 abstract interfaces
       emotionPort.py
       intentAndNeedsPort.py
       generationPort.py
       retrievalPort.py
       vectorStorePort.py
       verificationPort.py
    entities/            # Pure business objects
    valueObjects/        # Immutable results
    enums/               # Domain constants
    services/            # Business logic
    context/             # Context data

 adapters/                # Implement ports
    analysis/            # BERT, BART (FastAPI microservices)
    generation/          # OpenAI, local (external API)
    verification/        # 3 verifiers (FastAPI microservice)
    retrieval/           # ChromaDB, MiniLM, PDF (FastAPI microservice)

 application/             # Orchestrate through ports
```

```
    chatOrchestrator.py
    embeddingOrchestrator.py

 presentation/           # Expose via HTTP
    app.py
    routers/

 tests/                  # Validate each layer
     unit/
     integration/
     e2e/
```

6 Neurodialogue as a Microservice Architecture

The entire Neurodialogue system is designed as a self-contained microservice that can be integrated with any backend framework (Django, FastAPI, Flask, Node.js, etc.) through well-defined APIs.

6.1 Microservice Architecture Overview

Neurodialogue operates as a complete, self-contained microservice with the following characteristics :

— **Framework Agnostic** : Can be called by any backend framework (Django, FastAPI, Flask, Express, etc.) through HTTP/REST or gRPC.

— **Clean Architecture Inside** : Internally maintains hexagonal architecture with strict domain isolation.

— **AI Model Encapsulation** : All AI models (BERT, BART, MiniLM) are internal adapters, not external services.

— **Single Deployment Unit** : The entire system deploys as one cohesive microservice.

6.2 Communication with External Frameworks

External Framework	**Integration Method**
Django	HTTP calls to Neurodialogue API endpoints
FastAPI	Direct gRPC communication or REST API
Node.js / Express	RESTful API consumption
Mobile Apps (iOS / Android)	Secure API calls with authentication
Message Queues	Asynchronous communication via RabbitMQ or Kafka

6.3 Internal Architecture (Within the Microservice)

Inside the Neurodialogue microservice, the hexagonal architecture ensures :

— **Domain Isolation** : Business logic is completely separated from AI models and infrastructure.

— **Port-Based Communication** : All internal communication happens through well-defined abstract ports.

— **Testability** : Each component can be tested independently.

— **Model Swappability** : AI models can be replaced without affecting the external API contract.

6.4 Benefits of This Architecture

Aspect	**Benefit**
Technology Independence	Works with any backend framework (Python, JavaScript, Java, etc.)
Scalability	Neurodialogue can be scaled independently from the main application
Maintainability	Internal hexagonal architecture simplifies updates and maintenance
Deployment Flexibility	Can be deployed on-premise, in the cloud, or as a managed service
Team Autonomy	Backend teams and AI teams can work independently
Testing	API contract testing ensures compatibility with any frontend/backend

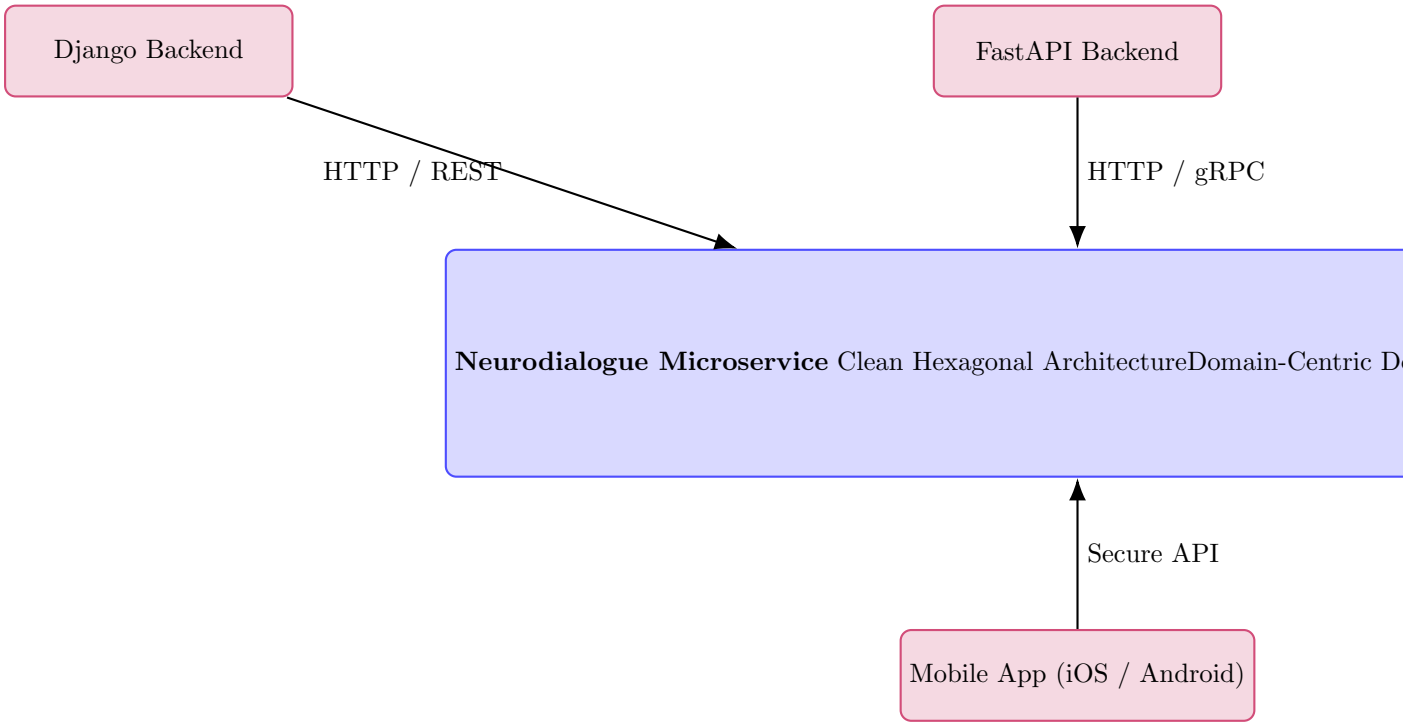


FIGURE 2 – Neurodialogue as an Independent Microservice

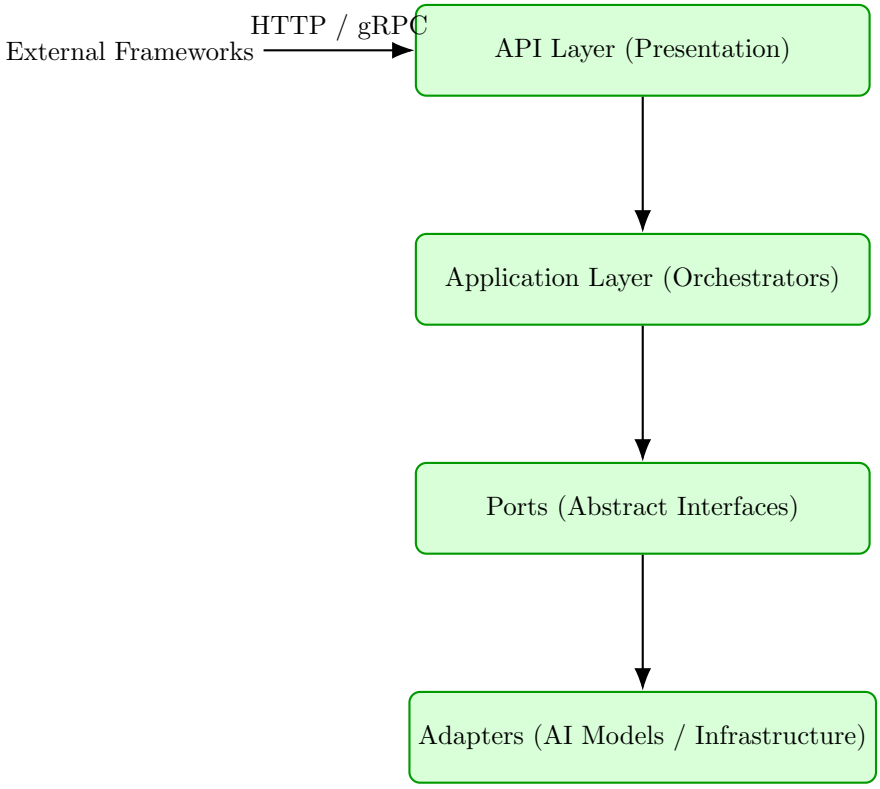


FIGURE 3 – Internal Layers of the Neurodialogue Microservice

6.5 Example Integration with Different Frameworks

Listing 4 – Calling Neurodialogue from Django (Python)

```python
import requests

def chat_with_ai(user_message):
    response = requests.post(
        "http://neurodialogue-service:8000/api/chat",
        json={"message": user_message, "user_id": request.user.id}
    )
    return response.json()["response"]
```

Listing 5 – Calling Neurodialogue from Node.js

```javascript
const axios = require('axios');

async function chatWithAI(message) {
    const response = await axios.post(
        'http://neurodialogue-service:8000/api/chat',
        { message: message, userId: req.session.userId }
    );
    return response.data.response;
}
```

Listing 6 – Calling Neurodialogue from FastAPI (as client)

```python
import httpx

@app.post("/chat")
async def chat_endpoint(user_message: str):
    async with httpx.AsyncClient() as client:
        response = await client.post(
            "http://neurodialogue-service:8000/api/chat",
            json={"message": user_message}
        )
        return response.json()
```

6.6 API Contract

The Neurodialogue microservice exposes a clean, versioned API :

Endpoint	Method	Description
/api/v1/chat	POST	Process a chat message and return AI response
/api/v1/emotions	POST	Analyze emotions in text
/api/v1/verify	POST	Verify response safety and empathy
/api/v1/health	GET	Health check endpoint
/api/v1/metrics	GET	Monitoring and metrics

This architecture ensures that Neurodialogue remains a self-contained, independently deployable microservice while maintaining its clean hexagonal internal structure, ready to serve any backend framework or frontend application.

7 Verification Rule

To verify that the hexagonal architecture is respected, run :

```
grep -r "from␣adapters" domain/
grep -r "from␣adapters" application/
```

These two commands must return zero results. The domain and application must never directly import from the adapters.

8 Conclusion

Hexagonal architecture provides Neurodialogue with :
— Complete isolation of business logic from technical concerns
— Easy testing through mocking at port boundaries
— Flexibility to change models, databases, and frameworks
— Clear separation of concerns and maintainable code
— Independent deployment of AI models as FastAPI microservices
— Scalability and team autonomy

The ports and adapters pattern ensures that the core of Neurodialogue remains stable and technology-agnostic, while the periphery can evolve with technological advances.